

LungLens

See what the AI sees. Explainable chest X-ray screening for earlier pneumonia detection.

Source code listing · UnivaBio Hackathon, AI for Human Health

Language: Python · 9 files · 836 lines (including comments)

Test-set result: 90.1% accuracy, 98.5% sensitivity, 76.1% specificity, AUC 0.955 on 624 unseen X-rays.

File	Purpose	Lines
README.md	Project overview and how to run it	91
requirements.txt	Python libraries the project needs	8
prepare_data.py	Step 1: download and unpack the X-ray dataset	93
dataset.py	Loading images and the patient-level train/validation split	68
network.py	The neural network and image preprocessing	114
train.py	Step 2: fine-tune the network	139
evaluate.py	Step 3: measure the model on the unseen test set	114
gradcam.py	Explainability: Grad-CAM heatmaps	75
app.py	Step 4: the interactive Streamlit web app	134

1. README.md

Project overview and how to run it

```
1  # 🦾 LungLens
2
3  **See what the AI sees.** Explainable chest X-ray screening for earlier pneumonia detection.
4
5  LungLens is a web app that looks at a chest X-ray and, in a few seconds, says how
6  likely it is that the patient has pneumonia. It also draws a **Grad-CAM heatmap**
7  over the X-ray showing which regions drove that decision, so a doctor can check
8  the AI's reasoning instead of trusting a black box.
9
10 Built for the UnivaBio hackathon (theme: AI for Human Health).
11
12 > ⚠️ **Screening aid, not a diagnosis.** LungLens is a student research prototype.
13 > It must not be used to make medical decisions.
14
15 ## Results
16
17 Measured on the 624 test X-rays, which were never used for training or tuning:
18
19 | Metric | Value |
20 |---|---|
21 | Accuracy | 90.1% |
22 | Sensitivity (pneumonia cases caught) | 98.5% |
23 | Specificity (healthy X-rays correctly cleared) | 76.1% |
24 | AUC | 0.955 |
25
26 ![Grad-CAM examples](results/gradcam_examples.png)
27
28 ## How it works
29
30 1. **Data:** 5,856 chest X-rays of children aged 1 to 5, labelled NORMAL or
31   PNEUMONIA by physicians (Kermay et al., *Cell* 2018, CC BY 4.0).
32 2. **Model:** a DenseNet121 convolutional neural network, pretrained by the
33   [TorchXRyVision](https://github.com/mlmed/torchxrayvision) project on
34   hundreds of thousands of adult chest X-rays, with a new final layer that
35   outputs one number: the probability of pneumonia.
36 3. **Training:** only the last dense block and the new layer are fine-tuned,
37   for 5 epochs on a CPU. Validation patients are held out by patient ID, the
38   loss is weighted for the 3:1 class imbalance, and the alert threshold is
39   chosen on validation data to balance sensitivity and specificity.
40 4. **Explainability:** Grad-CAM uses the gradients flowing into the network's
41   last 7x7 feature maps to score how much each region of the X-ray mattered.
42 5. **App:** a Streamlit page where you upload an X-ray (or pick a sample) and see
43   the verdict, probability and heatmap side by side.
44
45 ## Project files
46
47 | File | What it does |
48 |---|---|
49 | `prepare_data.py` | Step 1. Downloads the dataset and unpacks it into image folders |
50 | `dataset.py` | Loads images and makes the patient-level train/validation split |
51 | `network.py` | The neural network and the image preprocessing |
52 | `train.py` | Step 2. Fine-tunes the network and saves `models/lunglens.pt` |
53 | `evaluate.py` | Step 3. Measures the model on the test set, saves charts to `results/` |
54 | `gradcam.py` | Builds the Grad-CAM heatmap and the coloured overlay |
55 | `app.py` | Step 4. The interactive web app |
56 | `samples/` | Six test-set X-rays for trying the app |
57 | `docs/` | One-page description, code PDF, demo video script, code walkthrough |
58
59 ## Run it yourself
60
61 You need Python 3.10 or newer.
62
63 ```bash
64 pip install -r requirements.txt
65
66 # Just run the app (uses the trained model in models/):
67 streamlit run app.py
68
69 # Or reproduce everything from scratch:
70 python prepare_data.py # downloads about 1.2 GB
71 python train.py # about 85 minutes on a 4-core CPU
72 python evaluate.py
73 streamlit run app.py
```

```

74
75
76 ## Limitations
77
78 - Trained on young children from a single hospital in China; performance on
79   adults or on X-rays from other hospitals and machines is unknown.
80 - Only two classes (normal and pneumonia). It does not detect other conditions.
81 - Grad-CAM heatmaps are coarse (7x7 grid) and show where the model looked,
82   not a medical outline of the infection.
83
84 ## Credits
85
86 - Dataset: Kermany, D. S. et al. "Identifying Medical Diagnoses and Treatable
87   Diseases by Image-Based Deep Learning." Cell 172(5), 2018.
88 - Pretrained weights: Cohen, J. P. et al. "TorchXRyVision: A library of chest
89   X-ray datasets and models." MIDL 2022.
90 - Grad-CAM: Selvaraju, R. R. et al. "Grad-CAM: Visual Explanations from Deep
91   Networks via Gradient-based Localization." ICCV 2017.

```

2. requirements.txt

Python libraries the project needs

```
1 torch>=2.2
2 torchvision>=0.17
3 torchxrayvision>=1.2
4 streamlit>=1.40
5 scikit-learn>=1.3
6 matplotlib>=3.8
7 pillow>=10.0
8 numpy>=1.26
```

3. prepare_data.py

Step 1: download and unpack the X-ray dataset

```
1  """
2  prepare_data.py - Step 1 of LungLens: download and unpack the chest X-ray dataset.
3
4  The dataset
5  -----
6  "Labeled Optical Coherence Tomography (OCT) and Chest X-Ray Images for
7  Classification" by Kermany et al. (Cell, 2018). It contains 5,856 chest
8  X-rays of children aged 1 to 5 from Guangzhou Women and Children's Medical
9  Center, each labelled NORMAL or PNEUMONIA by expert physicians.
10
11  Google hosts a copy of it (used in the official Keras tutorial) as TFRecord
12  files, so we can download it without a Kaggle account.
13
14  What this script does
15  -----
16  1. Downloads four files: train/test images and train/test file paths.
17  2. Reads them record by record. Each "images" record is one JPEG file, and the
18  matching "paths" record is its original file name, e.g.
19  "/PNEUMONIA/person1_bacteria_1.jpeg". The folder name is the label.
20  3. Writes every image back out as a normal .jpeg file in:
21      data/chest_xray/train/NORMAL/...
22      data/chest_xray/train/PNEUMONIA/...
23      data/chest_xray/test/NORMAL/...
24      data/chest_xray/test/PNEUMONIA/...
25
26  Run it once: python prepare_data.py
27  """
28
29  import struct
30  import urllib.request
31  from pathlib import Path
32
33  BASE_URL = "https://storage.googleapis.com/download.tensorflow.org/data/ChestXRay2017"
34  RAW_DIR = Path("data/raw") # where the downloaded TFRecord files go
35  OUT_DIR = Path("data/chest_xray") # where the unpacked .jpeg files go
36
37
38  def download(split: str, name: str) -> Path:
39  """Download one TFRecord file (skips it if we already have it)."""
40  target = RAW_DIR / split / f"{name}.tfrec"
41  if not target.exists():
42      target.parent.mkdir(parents=True, exist_ok=True)
43      url = f"{BASE_URL}/{split}/{name}.tfrec"
44      print(f"Downloading {url} ...")
45      urllib.request.urlretrieve(url, target)
46  return target
47
48
49  def read_tfrecord(path: Path):
50  """
51  Yield the raw bytes of every record in a TFRecord file.
52
53  A TFRecord file is just a list of records stored one after another.
54  Each record is laid out as:
55      8 bytes - length of the data (little-endian unsigned integer)
56      4 bytes - checksum of the length (we skip it)
57      N bytes - the data itself
58      4 bytes - checksum of the data (we skip it)
59  Reading it ourselves means we do not need to install TensorFlow.
60  """
61  with open(path, "rb") as f:
62      while True:
63          header = f.read(12)
64          if len(header) < 12: # reached the end of the file
65              return
66          length = struct.unpack("<Q", header[:8])[0]
67          data = f.read(length)
68          f.read(4) # skip the data checksum
69          yield data
70
71
72  def unpack(split: str) -> None:
73  """Turn one split (train or test) into a folder of labelled .jpeg files."""
```

```

74     images = read_tfrecord(download(split, "images"))
75     paths = read_tfrecord(download(split, "paths"))
76
77     counts = {"NORMAL": 0, "PNEUMONIA": 0}
78     # The two files are in the same order, so we can walk them side by side.
79     for jpeg_bytes, path_bytes in zip(images, paths):
80         original = Path(path_bytes.decode())      # e.g. ./PNEUMONIA/person1_bacteria_1.jpeg
81         label = original.parent.name              # "NORMAL" or "PNEUMONIA"
82         out = OUT_DIR / split / label / original.name
83         out.parent.mkdir(parents=True, exist_ok=True)
84         out.write_bytes(jpeg_bytes)
85         counts[label] += 1
86
87     print(f"{split}: {counts['NORMAL']} normal, {counts['PNEUMONIA']} pneumonia")
88
89
90 if __name__ == "__main__":
91     for split in ["train", "test"]:
92         unpack(split)
93     print(f"Done. Images are in {OUT_DIR}/")

```

4. dataset.py

Loading images and the patient-level train/validation split

```
1  """
2  dataset.py - Loading the X-ray images and splitting them fairly.
3
4  Why a "patient-level" split?
5  -----
6  Some children in the dataset have several X-rays. If one X-ray of a child
7  lands in the training set and another in the validation set, the model can
8  "recognise the child" rather than the disease, and our validation score
9  would look better than it really is. So we split by patient: all of a
10 child's X-rays go to the same side.
11 """
12
13 import random
14 from pathlib import Path
15
16 from PIL import Image
17 from torch.utils.data import Dataset
18
19 DATA_DIR = Path("data/chest_xray")
20 LABELS = {"NORMAL": 0, "PNEUMONIA": 1}
21
22
23 def patient_id(filename: str) -> str:
24     """
25     Work out which child an image belongs to from its file name:
26     person1000_bacteria_2931.jpeg -> "person1000"
27     IM-0115-0001.jpeg           -> "IM-0115"
28     NORMAL2-IM-1326-0001.jpeg   -> "NORMAL2-IM-1326"
29     """
30     stem = Path(filename).stem
31     if stem.startswith("person"):
32         return stem.split("_")[0]
33     return stem.rsplit("-", 1)[0]
34
35
36 def list_images(split: str):
37     """Return a list of (image_path, label) pairs for "train" or "test"."""
38     items = []
39     for class_name, label in LABELS.items():
40         for path in sorted((DATA_DIR / split / class_name).glob("*.jpeg")):
41             items.append((path, label))
42     return items
43
44
45 def train_val_split(items, val_fraction: float = 0.1, seed: int = 42):
46     """Hold out about 10% of the patients for validation."""
47     patients = sorted({patient_id(p.name) for p, _ in items})
48     random.Random(seed).shuffle(patients) # fixed seed = same split every run
49     val_patients = set(patients[: int(len(patients) * val_fraction)])
50     train = [(p, y) for p, y in items if patient_id(p.name) not in val_patients]
51     val = [(p, y) for p, y in items if patient_id(p.name) in val_patients]
52     return train, val
53
54
55 class XrayDataset(Dataset):
56     """A PyTorch Dataset: given an index, returns (image tensor, label)."""
57
58     def __init__(self, items, transform):
59         self.items = items
60         self.transform = transform
61
62     def __len__(self):
63         return len(self.items)
64
65     def __getitem__(self, index):
66         path, label = self.items[index]
67         image = Image.open(path).convert("RGB")
68         return self.transform(image), float(label)
```

5. network.py

The neural network and image preprocessing

```
1  """
2  network.py - The LungLens neural network and image preprocessing.
3
4  Big idea: transfer learning
5  -----
6  Training a good image model from scratch needs hundreds of thousands of
7  images. We only have about 5,000. So we start from a DenseNet121 network
8  that the open-source TorchXRayVision project already trained on hundreds of
9  thousands of (adult) chest X-rays from public hospital datasets
10 (NIH, CheXpert, MIMIC, PadChest and others). That network has already learned
11 what ribs, lungs, the heart and cloudy patches look like.
12
13 We keep its "feature extractor" (the convolutional layers that turn an image
14 into a 7x7 grid of 1024 features) and replace its final layer with our own
15 single output: "how likely is pneumonia?". Training then only has to adapt
16 the last part of the network to our task (see train.py).
17 """
18
19 import torch
20 import torch.nn as nn
21 import torch.nn.functional as F
22 import torchxrayvision as xrv
23 from PIL import Image
24 from torchvision import transforms
25
26 IMAGE_SIZE = 224 # the pretrained network expects 224x224 images
27 CLASS_NAMES = ["NORMAL", "PNEUMONIA"]
28
29
30 class LungLensNet(nn.Module):
31     """DenseNet121 feature extractor + one output neuron for pneumonia."""
32
33     def __init__(self, pretrained: bool = True):
34         super().__init__()
35         # Load the chest-X-ray-pretrained DenseNet121. The weights file
36         # (about 28 MB) is downloaded from GitHub the first time.
37         base = xrv.models.DenseNet(weights="densenet121-res224-all" if pretrained else None)
38
39         # The convolutional part: image (1x224x224) -> features (1024x7x7).
40         # Each of the 7x7 cells "looks at" one region of the X-ray.
41         self.features = base.features
42
43         # Our new final layer: 1024 numbers -> 1 number (the pneumonia score).
44         self.classifier = nn.Linear(1024, 1)
45
46     def forward(self, x: torch.Tensor, return_features: bool = False):
47         # 1. Extract a 7x7 grid of features from the X-ray.
48         feature_maps = F.relu(self.features(x)) # shape: (batch, 1024, 7, 7)
49         # 2. Average each feature over the whole image ("global average pooling").
50         pooled = F.adaptive_avg_pool2d(feature_maps, 1).flatten(1) # (batch, 1024)
51         # 3. Combine the 1024 averages into one score. This is a "logit":
52         # any real number; sigmoid(logit) turns it into a probability 0..1.
53         logit = self.classifier(pooled).squeeze(1) # (batch,)
54         if return_features:
55             # Grad-CAM (gradcam.py) needs the 7x7 feature maps as well.
56             return logit, feature_maps
57         return logit
58
59     def freeze_early_layers(self) -> None:
60         """
61         Freeze everything except the last dense block, so training only
62         updates the deepest layers and our new classifier. The early layers
63         detect general things like edges and textures, which are already
64         good. This also makes training fast enough to run on a laptop CPU.
65         """
66         for name, layer in self.features.named_children():
67             trainable = name in ("denseblock4", "norm5")
68             for p in layer.parameters():
69                 p.requires_grad = trainable
70
71
72 def to_model_range(img: torch.Tensor) -> torch.Tensor:
73     """
```

```

74     The pretrained network expects pixel values from -1024 to +1024
75     (that mimics the range of real X-ray scanner values).
76     ToTensor() gives us 0..1, so we stretch it to -1024..+1024.
77     """
78     return img * 2048.0 - 1024.0
79
80
81     # Preprocessing used for prediction (and for validation/testing):
82     # grayscale -> resize to 224x224 -> tensor -> rescale to the model's range.
83     eval_transform = transforms.Compose([
84         transforms.Grayscale(num_output_channels=1),
85         transforms.Resize((IMAGE_SIZE, IMAGE_SIZE)),
86         transforms.ToTensor(),
87         transforms.Lambda(to_model_range),
88     ])
89
90     # Preprocessing used during training adds small random changes
91     # ("data augmentation") so the model sees a slightly different version of
92     # each X-ray every time. This helps it generalise instead of memorising.
93     train_transform = transforms.Compose([
94         transforms.Grayscale(num_output_channels=1),
95         transforms.RandomResizedCrop(IMAGE_SIZE, scale=(0.8, 1.0), ratio=(0.9, 1.1)),
96         transforms.RandomRotation(10),                # tilt up to 10 degrees
97         transforms.ColorJitter(brightness=0.2, contrast=0.2), # vary exposure
98         transforms.ToTensor(),
99         transforms.Lambda(to_model_range),
100    ])
101
102
103    def load_model(checkpoint_path: str = "models/lunglens.pt"):
104        """Load the trained model and the decision threshold saved by train.py."""
105        checkpoint = torch.load(checkpoint_path, map_location="cpu")
106        model = LungLensNet(pretrained=False) # weights come from our checkpoint instead
107        model.load_state_dict(checkpoint["model_state"])
108        model.eval() # switch off training-only behaviour
109        return model, checkpoint["threshold"]
110
111
112    def preprocess(image: Image.Image) -> torch.Tensor:
113        """Turn a PIL image into a batch of one tensor ready for the model."""
114        return eval_transform(image.convert("RGB")).unsqueeze(0)

```

6. train.py

Step 2: fine-tune the network

```
1  """
2  train.py - Step 2 of LungLens: fine-tune the model on the X-rays.
3
4  Run: python train.py
5  Output: models/lunglens.pt (the trained weights + chosen decision threshold)
6          results/training_log.json
7
8  How training works, in one paragraph
9  -----
10 We show the model a batch of 32 X-rays, it guesses a pneumonia score for
11 each, and we measure how wrong the guesses are with a "loss" function.
12 Back-propagation then works out, for every adjustable number (weight) in the
13 network, which direction would make the loss smaller, and the optimiser
14 nudges each weight a little in that direction. One pass over all training
15 images is an "epoch". After each epoch we check the model on validation
16 images it never trains on, and keep the best version.
17 """
18
19 import json
20 import time
21 from pathlib import Path
22
23 import numpy as np
24 import torch
25 import torch.nn as nn
26 from sklearn.metrics import roc_auc_score
27 from torch.utils.data import DataLoader
28
29 from dataset import XrayDataset, list_images, train_val_split
30 from network import LungLensNet, eval_transform, train_transform
31
32 EPOCHS = 5
33 BATCH_SIZE = 32
34 SEED = 42
35
36
37 def predict_probabilities(model, loader):
38     """Run the model over a whole dataset and return (probabilities, labels)."""
39     model.eval()
40     probs, labels = [], []
41     with torch.no_grad(): # no learning here, so skip gradients
42         for images, y in loader:
43             probs.append(torch.sigmoid(model(images)).numpy())
44             labels.append(y.numpy())
45     return np.concatenate(probs), np.concatenate(labels)
46
47
48 def best_threshold(probs, labels):
49     """
50     A probability above the threshold means "flag as pneumonia".
51     Instead of blindly using 0.5 we pick the threshold that maximises
52     sensitivity + specificity on the validation set (Youden's J statistic),
53     so the model is balanced between missing sick children and raising
54     false alarms on healthy ones.
55     """
56     best_t, best_j = 0.5, -1.0
57     for t in np.linspace(0.05, 0.95, 91):
58         predicted = probs >= t
59         sensitivity = (predicted & (labels == 1)).sum() / (labels == 1).sum()
60         specificity = (~predicted & (labels == 0)).sum() / (labels == 0).sum()
61         if sensitivity + specificity - 1 > best_j:
62             best_t, best_j = float(t), sensitivity + specificity - 1
63     return best_t
64
65
66 def main():
67     torch.manual_seed(SEED)
68     Path("models").mkdir(exist_ok=True)
69     Path("results").mkdir(exist_ok=True)
70
71     # ---- Data -----
72     train_items, val_items = train_val_split(list_images("train"))
73     n_pneumonia = sum(y for _, y in train_items)
```

```

74     n_normal = len(train_items) - n_pneumonia
75     print(f"Training on {len(train_items)} images ({n_normal} normal, {n_pneumonia} pneumonia)")
76     print(f"Validating on {len(val_items)} images")
77
78     train_loader = DataLoader(XrayDataset(train_items, train_transform),
79                               batch_size=BATCH_SIZE, shuffle=True, num_workers=3)
80     val_loader = DataLoader(XrayDataset(val_items, eval_transform),
81                             batch_size=BATCH_SIZE, num_workers=3)
82
83     # ---- Model -----
84     model = LungLensNet(pretrained=True)
85     model.freeze_early_layers()
86
87     # The dataset has about 3x more pneumonia than normal images. Without a
88     # correction the model could score well by mostly saying "pneumonia".
89     # pos_weight < 1 makes each pneumonia example count less, so both
90     # classes matter equally in the loss.
91     loss_fn = nn.BCEWithLogitsLoss(pos_weight=torch.tensor(n_normal / n_pneumonia))
92
93     # AdamW optimiser. The new classifier layer starts from random numbers,
94     # so it gets a larger learning rate than the pretrained layers.
95     optimizer = torch.optim.AdamW([
96         {"params": [p for p in model.features.parameters() if p.requires_grad], "lr": 1e-4},
97         {"params": model.classifier.parameters(), "lr": 1e-3},
98     ], weight_decay=1e-4)
99     # Gradually lower the learning rate over training (cosine schedule).
100    scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=EPOCHS)
101
102    # ---- Training loop -----
103    log, best_auc = [], 0.0
104    for epoch in range(1, EPOCHS + 1):
105        model.train()
106        start, running_loss = time.time(), 0.0
107        for step, (images, labels) in enumerate(train_loader, 1):
108            optimizer.zero_grad() # clear last step's gradients
109            loss = loss_fn(model(images), labels.float())
110            loss.backward() # back-propagation
111            optimizer.step() # nudge the weights
112            running_loss += loss.item()
113            if step % 20 == 0:
114                print(f" epoch {epoch} step {step}/{len(train_loader)} loss {running_loss / step:.4
115f}", flush=True)
116            scheduler.step()
117
118        # Check how well we do on the validation patients.
119        val_probs, val_labels = predict_probabilities(model, val_loader)
120        val_auc = roc_auc_score(val_labels, val_probs)
121        entry = {"epoch": epoch, "train_loss": running_loss / len(train_loader),
122                "val_auc": float(val_auc), "minutes": (time.time() - start) / 60}
123        log.append(entry)
124        print(entry, flush=True)
125
126        # Keep the best model so far (highest validation AUC).
127        if val_auc > best_auc:
128            best_auc = val_auc
129            torch.save({"model_state": model.state_dict(),
130                        "threshold": best_threshold(val_probs, val_labels),
131                        "epoch": epoch, "val_auc": float(val_auc)},
132                      "models/lunglens.pt")
133            print(f" saved new best model (val AUC {val_auc:.4f})")
134
135    Path("results/training_log.json").write_text(json.dumps(log, indent=2))
136    print("Training finished.")
137
138
139    if __name__ == "__main__":
140        main()

```

7. evaluate.py

Step 3: measure the model on the unseen test set

```
1  """
2  evaluate.py - Step 3 of LungLens: measure the model honestly.
3
4  The 624 test X-rays were never used for training or for choosing the
5  threshold, so they show how the model would do on new children.
6
7  Run: python evaluate.py
8  Output: results/metrics.json, results/confusion_matrix.png,
9          results/roc_curve.png, results/gradcam_examples.png
10
11  The numbers that matter for screening
12  -----
13  - Sensitivity (recall): of the children who HAVE pneumonia, how many did we
14    flag? A screening tool must keep this high: a miss means a sick child is
15    sent home.
16  - Specificity: of the healthy children, how many did we correctly clear?
17    Low specificity means false alarms and extra work for doctors.
18  - AUC: how well the scores separate sick from healthy over every possible
19    threshold. 1.0 is perfect, 0.5 is a coin flip.
20  """
21
22  import json
23  import random
24
25  import matplotlib
26  matplotlib.use("Agg") # draw plots to files, no window needed
27  import matplotlib.pyplot as plt
28  import numpy as np
29  from PIL import Image
30  from sklearn.metrics import confusion_matrix, roc_auc_score, roc_curve
31  from torch.utils.data import DataLoader
32
33  from dataset import XrayDataset, list_images
34  from gradcam import gradcam, overlay
35  from network import eval_transform, load_model, preprocess
36  from train import predict_probabilities
37
38
39  def main():
40      model, threshold = load_model()
41      test_items = list_images("test")
42      loader = DataLoader(XrayDataset(test_items, eval_transform), batch_size=32, num_workers=3)
43      probs, labels = predict_probabilities(model, loader)
44      predicted = (probs >= threshold).astype(int)
45
46      # Confusion matrix: rows = truth, columns = prediction.
47      tn, fp, fn, tp = confusion_matrix(labels, predicted).ravel()
48      metrics = {
49          "test_images": int(len(labels)),
50          "threshold": threshold,
51          "accuracy": (tp + tn) / len(labels),
52          "sensitivity": tp / (tp + fn),
53          "specificity": tn / (tn + fp),
54          "precision": tp / (tp + fp),
55          "f1": 2 * tp / (2 * tp + fp + fn),
56          "auc": roc_auc_score(labels, probs),
57          "true_positive": tp, "false_negative": fn,
58          "true_negative": tn, "false_positive": fp,
59      }
60      metrics = {k: (round(float(v), 4) if isinstance(v, (float, np.floating)) else int(v) if isinstance
61 (v, np.integer) else v)
62                  for k, v in metrics.items()}
63      print(json.dumps(metrics, indent=2))
64      with open("results/metrics.json", "w") as f:
65          json.dump(metrics, f, indent=2)
66
67      # ---- Confusion matrix picture -----
68      fig, ax = plt.subplots(figsize=(4.2, 3.8))
69      matrix = np.array([[tn, fp], [fn, tp]])
70      ax.imshow(matrix, cmap="Blues")
71      for i in range(2):
72          for j in range(2):
73              ax.text(j, i, matrix[i, j], ha="center", va="center", fontsize=16,
```

```

74         color="white" if matrix[i, j] > matrix.max() / 2 else "black")
75     ax.set_xticks([0, 1], ["Normal", "Pneumonia"])
76     ax.set_yticks([0, 1], ["Normal", "Pneumonia"])
77     ax.set_xlabel("LungLens prediction")
78     ax.set_ylabel("Doctor's label")
79     ax.set_title("Test set (624 X-rays)")
80     fig.tight_layout()
81     fig.savefig("results/confusion_matrix.png", dpi=200)
82
83     # ---- ROC curve -----
84     fpr, tpr, _ = roc_curve(labels, probs)
85     fig, ax = plt.subplots(figsize=(4.2, 3.8))
86     ax.plot(fpr, tpr, linewidth=2, label=f"LungLens (AUC = {metrics['auc']:.3f})")
87     ax.plot([0, 1], [0, 1], "--", color="gray", label="Random guess")
88     ax.set_xlabel("False alarm rate (1 - specificity)")
89     ax.set_ylabel("Sensitivity")
90     ax.set_title("ROC curve, test set")
91     ax.legend(loc="lower right")
92     fig.tight_layout()
93     fig.savefig("results/roc_curve.png", dpi=200)
94
95     # ---- A few Grad-CAM examples -----
96     random.seed(0)
97     picks = random.sample([i for i in test_items if i[1] == 0], 2) + \
98         random.sample([i for i in test_items if i[1] == 1], 2)
99     fig, axes = plt.subplots(2, 4, figsize=(12, 6.4))
100    for col, (path, label) in enumerate(picks):
101        image = Image.open(path).convert("RGB")
102        probability, heat = gradcam(model, preprocess(image), threshold)
103        axes[0, col].imshow(image, cmap="gray")
104        axes[0, col].set_title(f"Doctor: {'Pneumonia' if label else 'Normal'}")
105        axes[1, col].imshow(overlay(image, heat))
106        axes[1, col].set_title(f"LungLens: {probability:.0%} pneumonia")
107        for row in range(2):
108            axes[row, col].axis("off")
109    fig.tight_layout()
110    fig.savefig("results/gradcam_examples.png", dpi=150)
111    print("Saved plots to results/")
112
113
114    if __name__ == "__main__":
115        main()

```

8. gradcam.py

Explainability: Grad-CAM heatmaps

```
1  """
2  gradcam.py - "See what the AI sees": Grad-CAM heatmaps.
3
4  The problem
5  -----
6  A neural network gives an answer but not a reason. A doctor should never
7  trust a "black box", so LungLens shows WHERE in the X-ray the model found
8  the evidence for its answer.
9
10 Grad-CAM in four steps (Selvaraju et al., 2017)
11 -----
12 1. Run the X-ray through the network and keep the last 7x7 grid of
13    feature maps (1024 maps, each 7x7). Each cell covers one patch of the
14    X-ray, roughly 32x32 pixels.
15 2. Ask: "if this feature map got a bit stronger, how much would the
16    answer change?" That is the gradient of the score with respect to
17    the feature map, computed by back-propagation.
18 3. Average each map's gradient into one importance weight, then add up
19    all 1024 maps multiplied by their weights. Keep only positive values
20    (ReLU): we want regions that SUPPORT the answer.
21 4. Stretch the 7x7 result up to the X-ray's size and colour it:
22    red = strong evidence, blue = little evidence.
23 """
24
25 import numpy as np
26 import torch
27 import torch.nn.functional as F
28 from matplotlib import colormaps
29 from PIL import Image
30
31
32 def gradcam(model, image_tensor: torch.Tensor, threshold: float = 0.5):
33     """
34     Returns (probability_of_pneumonia, heatmap) where heatmap is a 224x224
35     numpy array with values from 0 (ignored) to 1 (most important).
36     """
37     model.eval()
38     # Step 1: forward pass, keeping the feature maps.
39     logit, feature_maps = model(image_tensor, return_features=True)
40     probability = torch.sigmoid(logit).item()
41
42     # Explain the answer the model actually gives. For a "pneumonia" answer
43     # we explain the pneumonia score; for a "normal" answer we flip the
44     # sign, so the map shows the evidence for "normal" instead.
45     target = logit if probability >= threshold else -logit
46
47     # Step 2: gradient of the target score with respect to the feature maps.
48     gradients = torch.autograd.grad(target.sum(), feature_maps)[0] # (1, 1024, 7, 7)
49
50     # Step 3: one weight per feature map, then a weighted sum of the maps.
51     weights = gradients.mean(dim=(2, 3), keepdim=True) # (1, 1024, 1, 1)
52     cam = F.relu(weights * feature_maps).sum(dim=1, keepdim=True) # (1, 1, 7, 7)
53
54     # Step 4: upscale to 224x224 and rescale to 0..1.
55     cam = F.interpolate(cam, size=image_tensor.shape[-2:], mode="bilinear", align_corners=False)
56     cam = cam.squeeze().detach().numpy()
57     cam = (cam - cam.min()) / (cam.max() - cam.min() + 1e-8)
58     return probability, cam
59
60
61 def overlay(image: Image.Image, heatmap: np.ndarray, opacity: float = 0.6) -> Image.Image:
62     """
63     Colour the heatmap (blue -> red) and blend it on top of the X-ray.
64     Each pixel's colour strength follows its heat, so unimportant areas
65     stay a clear X-ray and only the important regions light up.
66     """
67     xray = np.asarray(image.convert("RGB")).astype(float) / 255.0
68     # Resize the 224x224 heatmap to the original X-ray size.
69     heat = Image.fromarray(np.uint8(heatmap * 255)).resize(image.size, Image.BILINEAR)
70     heat = np.asarray(heat).astype(float) / 255.0
71     # The "jet" colour map turns 0..1 into blue..green..yellow..red.
72     coloured = colormaps["jet"](heat)[: , :, :3]
73     alpha = (heat * opacity)[: , :, None] # per-pixel blend strength
```

```
74     blended = xray * (1 - alpha) + coloured * alpha
75     return Image.fromarray(np.uint8(blended * 255))
```

9. app.py

Step 4: the interactive Streamlit web app

```
1  """
2  app.py - The LungLens web app (Step 4).
3
4  Run: streamlit run app.py
5  Then open the link it prints (usually http://localhost:8501).
6
7  What the user does
8  -----
9  1. Uploads a chest X-ray (or picks one of the sample images).
10 2. LungLens shows its screening result: the chance of pneumonia and
11   whether that is above the alert threshold.
123. Next to the X-ray it shows a Grad-CAM heatmap of where the model
13   "looked" when deciding, so a clinician can check the reasoning.
14  """
15
16 import json
17 from pathlib import Path
18
19 import numpy as np
20 import streamlit as st
21 from PIL import Image
22
23 from gradcam import gradcam, overlay
24 from network import load_model, preprocess
25
26 st.set_page_config(page_title="LungLens", page_icon="🦠", layout="wide")
27
28
29 @st.cache_resource          # load the model once, not on every click
30 def get_model():
31     return load_model("models/lunglens.pt")
32
33
34 def looks_like_xray(image: Image.Image) -> bool:
35     """
36     A simple sanity check: X-rays are grayscale, so the red, green and
37     blue values of each pixel are almost identical. A colourful photo
38     is probably not an X-ray.
39     """
40     pixels = np.asarray(image.convert("RGB").resize((128, 128))).astype(float)
41     colourfulness = np.abs(pixels[:, :, 0] - pixels[:, :, 1]).mean() + \
42                     np.abs(pixels[:, :, 1] - pixels[:, :, 2]).mean()
43     return colourfulness < 10
44
45
46 model, threshold = get_model()
47 metrics_file = Path("results/metrics.json")
48 metrics = json.loads(metrics_file.read_text()) if metrics_file.exists() else None
49
50 # ----- Sidebar
51 with st.sidebar:
52     st.title("🦠 LungLens")
53     st.write("**See what the AI sees.** Explainable chest X-ray screening "
54             "for earlier pneumonia detection.")
55     st.subheader("How it works")
56     st.markdown(
57         "1. A **DenseNet121** neural network, pretrained on hundreds of thousands of "
58         "chest X-rays, was fine-tuned on 4,694 children's X-rays.\n"
59         "2. It outputs the **probability of pneumonia**.\n"
60         "3. **Grad-CAM** traces the decision back to the image and "
61         "highlights the regions that drove it.")
62     if metrics:
63         st.subheader(f"Tested on {metrics['test_images']} unseen X-rays")
64         c1, c2 = st.columns(2)
65         c1.metric("Sensitivity", f"{metrics['sensitivity']:.1%}")
66         c2.metric("Specificity", f"{metrics['specificity']:.1%}")
67         c1.metric("Accuracy", f"{metrics['accuracy']:.1%}")
68         c2.metric("AUC", f"{metrics['auc']:.3f}")
69         st.caption("Sensitivity: share of pneumonia cases caught. "
70                 "Specificity: share of healthy X-rays correctly cleared.")
71
72 # ----- Main
73 st.title("LungLens: pneumonia screening you can see")
```

```

74 st.warning("⚠️ **Screening aid, not a diagnosis.** LungLens is a student research "
75            "prototype trained on one public dataset of children's X-rays. It must not "
76            "be used to make medical decisions. Always consult a qualified doctor.")
77
78 source = st.radio("Choose an X-ray", ["Try a sample", "Upload my own"], horizontal=True)
79 image = None
80 if source == "Upload my own":
81     uploaded = st.file_uploader("Chest X-ray image (JPG or PNG, front view)",
82                                type=["jpg", "jpeg", "png"])
83     if uploaded:
84         image = Image.open(uploaded)
85 else:
86     samples = sorted(Path("samples").glob("*.jpeg"))
87     choice = st.selectbox("Sample X-rays (from the test set, never seen in training)",
88                          samples, format_func=lambda p: p.stem.split("_")[0] + " example " + p.stem.split("_")[1])
89     image = Image.open(choice)
90
91 if image is not None:
92     image = image.convert("RGB")
93     if not looks_like_xray(image):
94         st.error("This image is in colour, so it is probably not a chest X-ray. "
95                "The result below is unlikely to mean anything.")
96
97 with st.spinner("Analysing the X-ray..."):
98     probability, heatmap = gradcam(model, preprocess(image), threshold)
99     flagged = probability >= threshold
100
101 # Three columns: the X-ray, the heatmap, and the result with its explanation.
102 col_xray, col_heat, col_result = st.columns([1, 1, 1.1])
103
104 with col_result:
105     if flagged:
106         st.error(f"### 🚫 Signs of pneumonia\n"
107                f"Pneumonia probability **{probability:.0%}**. Recommend review by a doctor.")
108     else:
109         st.success(f"### 🟢 No signs of pneumonia\n"
110                  f"Pneumonia probability **{probability:.0%}**.")
111     st.progress(probability, text=f"LungLens raises an alert at {threshold:.0%} or more")
112     opacity = st.slider("Heatmap strength", 0.0, 1.0, 0.6, 0.05)
113     with st.expander("How do I read the heatmap?", expanded=True):
114         st.markdown(
115             f"- The heatmap explains the answer LungLens gave "
116             f"({{'pneumonia' if flagged else 'normal'}}).\n"
117             "- **Red and yellow** areas are where the network found the strongest "
118             "evidence; uncoloured areas mattered little.\n"
119             "- In pneumonia, infected lung tissue fills with fluid and shows up as "
120             "white, cloudy patches (**consolidation**). A trustworthy result "
121             "highlights areas inside the lungs.\n"
122             "- If the heat sits on text labels, the image edges or outside the body, "
123             "the model may be using a shortcut: treat the result with extra caution.\n"
124             "- The map is built from a 7x7 grid, so it shows regions, not exact outlines.")
125
126 col_xray.image(image, caption="Original X-ray", use_container_width=True)
127 col_heat.image(overlay(image, heatmap, opacity),
128               caption="Grad-CAM: red = regions that drove the decision",
129               use_container_width=True)
130
131 else:
132     st.info("Upload a chest X-ray or pick a sample to begin.")
133
134 st.caption("Data: Kermay et al., Cell 2018 (CC BY 4.0). Pretrained weights: TorchXRayVision. "
135           "Built for the UnivaBio hackathon, AI for Human Health.")

```